

目 录

1 基于 Unity 的森林 VR 场景构建与展示	1
1.1 实验目的与场景主题	1
1.2 创建 Unity VR 模板项目	1
1.2.1 选择 VR 项目模板	1
1.2.2 项目环境确认	1
1.3 导入森林环境资源	3
1.3.1 从 Asset Store 获取资源	3
1.3.2 打开森林样例场景	3
1.4 粉色材质问题分析	5
1.4.1 渲染管线不兼容	5
1.4.2 植被材质的特殊要求	5
1.5 URP Shader 修复过程	6
1.5.1 修改 StandardNoCulling Shader	6
1.5.2 保留双面渲染和阴影	6
1.5.3 调整材质引用	7
1.6 接入 VR 视角	7
1.6.1 复制 XR Origin	7
1.6.2 调整出生点与相机冲突	8
1.7 运行展示与实验结果	8
1.8 本章小结	9
2 结论	10
参考文献	11
附录 人脸分析实验关键代码	12
页面入口与展示容器	12
图片表情识别脚本	13
年龄与性别估计脚本	16
摄像头实时追踪脚本	19
开源人脸识别模型训练代码	23

1 基于 Unity 的森林 VR 场景构建与展示

1.1 实验目的与场景主题

本章围绕 Unity 中的 VR 场景搭建过程展开，实验主题为森林环境。与前两章主要分析人脸识别模型和浏览器端推理流程不同，本章关注虚拟现实课程中的三维场景构建、资源导入、渲染兼容性处理和 VR 视角配置。实验目标是在 Unity 中创建一个可进入、可观察的森林 VR 场景，使用户能够通过 VR 视角进入自然环境，并在运行时观察地形、树木、草地和光照等空间元素。

森林主题适合作为 VR 场景实验对象。一方面，森林包含地形、植被、树叶、草丛、天空和光照等丰富视觉元素，可以较直观地展示三维环境的沉浸感；另一方面，植被资源通常依赖双面渲染、透明裁剪和阴影投射，对渲染管线兼容性有一定要求。因此，本实验不仅是简单地把模型拖入场景，还需要处理旧资源包导入 Unity 6 + Universal Render Pipeline 后出现的材质显示问题。

本章按照实际操作顺序记录实验过程：首先创建 Unity VR 模板项目，确认项目版本和渲染管线；然后从 Asset Store 导入 Fantasy Forest Environment Free Sample 资源包；接着分析并修复导入后场景大面积显示为粉色的问题；最后将 VR 模板场景中的 XR Origin 复制到森林样例场景中，完成 VR 视角的接入与运行展示。

1.2 创建 Unity VR 模板项目

1.2.1 选择 VR 项目模板

实验首先在 Unity Hub 中创建 VR 模板项目。与普通 3D 模板相比，VR 模板会预先配置 XR 相关依赖、输入系统、相机结构和交互组件，适合作为虚拟现实实验的起点。如果从普通 3D 项目开始，需要额外安装和配置 XR Plugin Management、XR Interaction Toolkit、Input System 等内容，配置过程更容易遗漏。使用 VR 模板可以把重点放在场景资源和运行展示上。

项目创建过程中，Unity 会生成模板场景、默认设置和示例资源。图 1.2 展示了等待项目创建的过程。项目创建完成后进入 Unity 编辑器，能够看到 VR 模板项目的默认场景与资源结构，如图 1.3 所示。

1.2.2 项目环境确认

为了判断后续导入的场景资源是否兼容，需要先确认当前 Unity 项目版本和渲染管线。本实验项目使用的 Unity 版本为 6000.3.14f1，渲染管线为 Universal Render Pipeline，URP 包版本为 17.3.0。其中，Unity 版本可从 ProjectSettings/ProjectVersion.txt

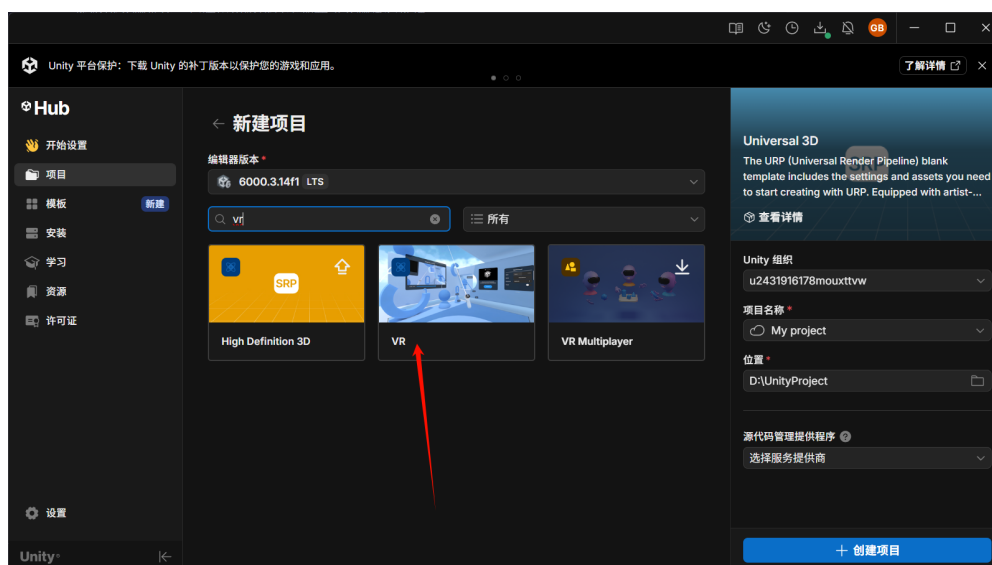


图 1.1 创建 Unity VR 模板项目



图 1.2 Unity VR 项目创建过程

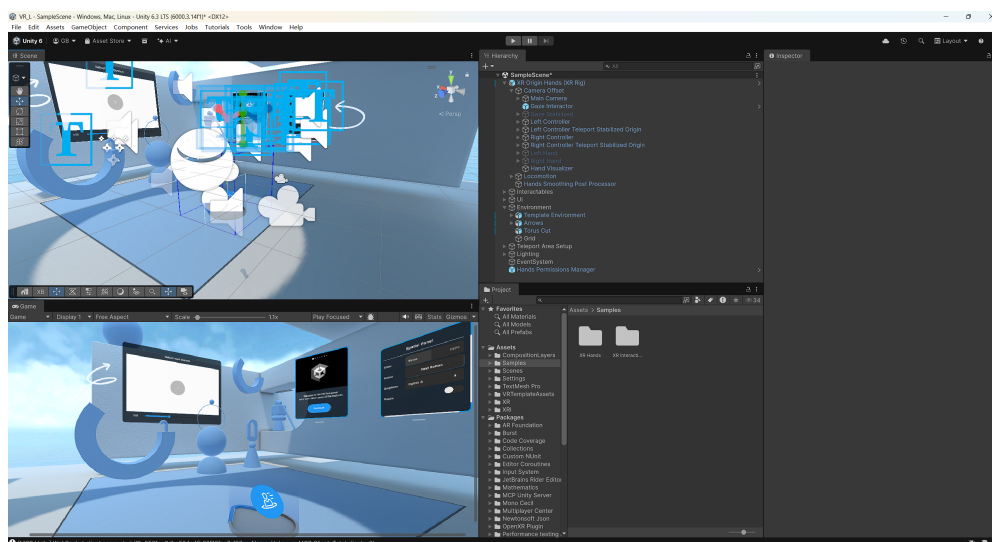


图 1.3 进入 Unity VR 模板项目

确认,URP 包版本可从 `Packages/manifest.json` 确认,当前渲染管线配置可从 `ProjectSettings/GraphicsSettings` 和 URP 配置资产中确认。

这些信息对后续问题定位非常重要。Unity 资源包往往会标注推荐版本和适配的渲染管线。如果资源包面向旧版 Built-in Render Pipeline，而当前项目使用 URP，则材质、Shader 和光照表现可能出现不兼容。实验中出现的粉色材质问题，正是由这一版本和渲染管线差异引起的。

1.3 导入森林环境资源

1.3.1 从 Asset Store 获取资源

项目创建完成后，实验进入 Unity Asset Store 查找森林环境资源。本次使用的资源包为 Fantasy Forest Environment Free Sample。该资源包提供了森林样例场景、地形、树木、草地、材质和贴图等内容，能够较快搭建出具有自然环境特征的 VR 展示空间。

资源导入后，项目中新增了 `Assets/Fantasy Forest Environment Free Sample` 目录。该目录包含样例场景、材质、模型和资源包自带的 Shader 文件。导入过程如图 1.6 所示。

1.3.2 打开森林样例场景

资源导入完成后,打开资源包提供的样例场景 `Assets/Fantasy Forest Environment Free Sample/Scenes/` 该场景已经包含地形、树木、草地和默认相机,理论上可以直接作为森林环境的展示基础。

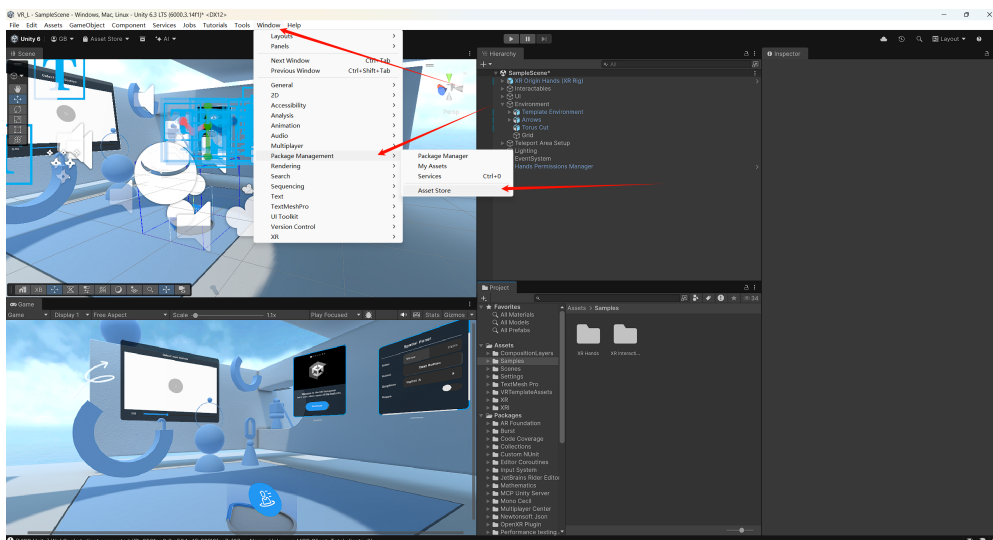


图 1.4 在 Unity 中前往 Asset Store

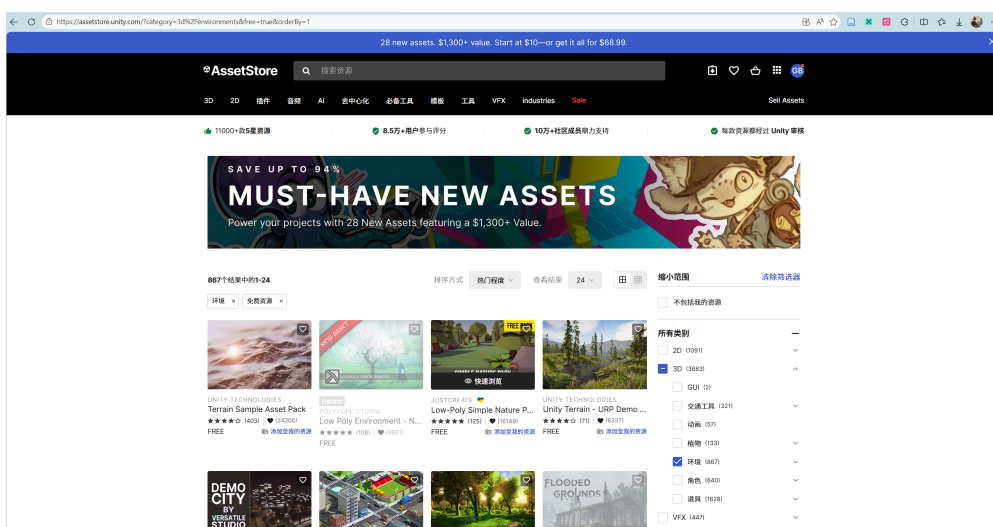


图 1.5 Asset Store 中的森林环境资源

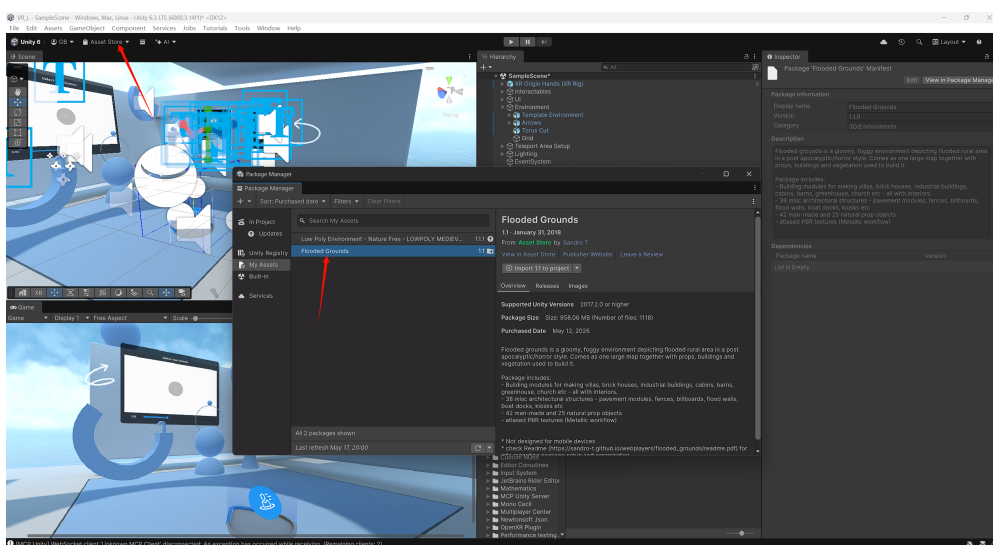


图 1.6 导入 Fantasy Forest Environment Free Sample 资源包

但是在当前 Unity 6 + URP 项目中打开样例场景后,模型和材质大面积显示为粉色,如图 1.7 所示。Unity 中物体显示为粉色,通常表示材质引用的 Shader 无法在当前渲染环境中正常编译或使用。也就是说,场景资源已经导入,但渲染管线不兼容导致它无法正确显示。

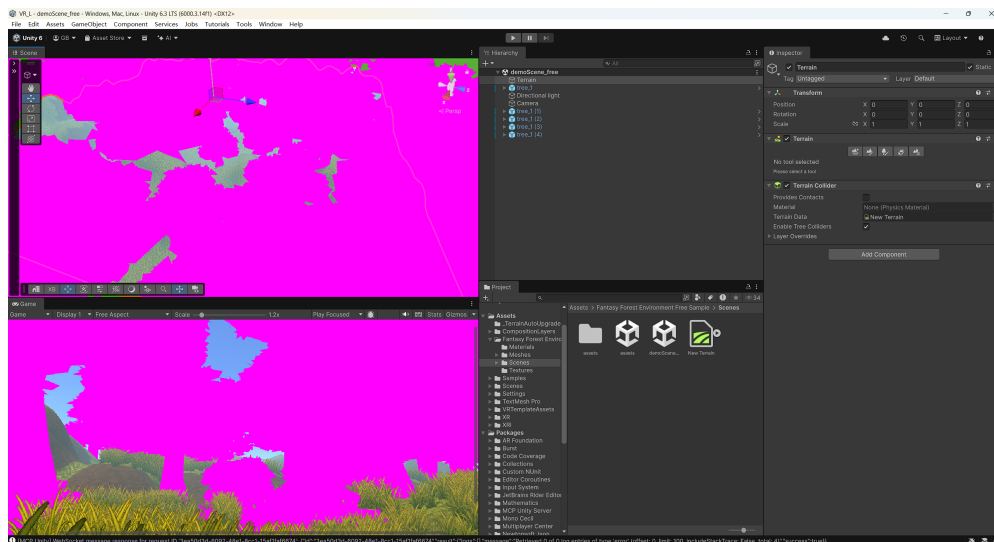


图 1.7 森林样例场景出现粉色材质

1.4 粉色材质问题分析

1.4.1 渲染管线不兼容

检查资源包信息后可以发现, Fantasy Forest Environment Free Sample 资源包推荐的 Unity 版本为 2021.3.9f1, 资源本身主要面向旧的 Built-in Render Pipeline。而当前实验项目使用 Unity 6 和 URP 17.3.0。Built-in 管线和 URP 在 Shader 写法、光照函数、渲染标签和材质兼容性方面存在差异, 因此旧资源包中的 Shader 不能直接在 URP 项目中稳定使用。

进一步检查材质文件后发现, dirt01.mat、grass01.mat 等材质仍引用旧的内置 Shader; grassmesh.mat、tree_branches.mat 等材质则引用资源包自带的 Fantasy Forest/StandardNoCulling Shader。原始 StandardNoCulling.shader 使用 Built-in 管线的 Surface Shader 写法, 例如 `#pragma surface surf StandardSpecular`。URP 不支持这种 Built-in Surface Shader, 因此这些材质在当前项目中无法正常渲染。

1.4.2 植被材质的特殊要求

森林资源中的草、树叶和树枝并不是普通不透明模型。为了让草片和树叶从正反两面都可见, 材质需要保留双面显示逻辑; 为了让贴图中透明区域不遮挡背景, 材

质还需要使用透明裁剪；为了让场景具备空间层次，植被对象还应能参与阴影投射。因此，修复时不能只把材质简单替换为普通 URP Lit Shader，还要保留原资源对双面渲染和 alpha 裁剪的需求。

基于这一判断，本实验没有选择降级 Unity 版本，也没有重新创建 Unity 2021 的 Built-in 项目。当前项目已经包含 VR 模板、URP、XR 和输入系统等依赖，直接降级项目存在较高风险。更合适的方式是在当前 Unity 6 + URP 项目中修复资源包的 Shader 和材质引用，使森林资源适配当前实验环境。

1.5 URP Shader 修复过程

1.5.1 修改 StandardNoCulling Shader

本次代码级修复集中在资源包自带的 Shader 文件：

```
1 | Assets/Fantasy Forest Environment Free Sample/StandardNoCulling.shader
```

修复思路是将旧 Built-in Surface Shader 改写为 URP 可用的 HLSL Shader，并继续使用原来的 Shader 名称 Fantasy Forest/StandardNoCulling。这样，已经引用该 Shader 的材质可以自动使用兼容版本，不需要逐个重建材质。

修复后的 Shader 保留了资源包原本需要的 `_MainTex`、`_Color`、`_Cutoff` 和 `_Glossiness` 属性。其中，`_MainTex` 用于基础贴图，`_Color` 用于颜色叠加，`_Cutoff` 用于透明裁剪阈值，`_Glossiness` 用于控制光滑度。SubShader 中增加 `RenderPipeline = UniversalPipeline` 标签，明确它面向 URP 使用。

核心渲染通道使用 `UniversalForward`，通过 URP 的 `Core.hlsl` 和 `Lighting.hlsl` 获取顶点变换、法线、主光源、阴影和雾效支持。片元阶段先采样基础贴图，并通过贴图 `alpha` 执行透明裁剪，再构造 `InputData` 与 `SurfaceData`，最后调用 `UniversalFragmentPBR` 输出光照结果。这样可以使草地、树叶和树干在 URP 中获得基本的 PBR 光照表现。

1.5.2 保留双面渲染和阴影

森林植被资源需要从正反两面都能看到，因此 Shader 中保留了 `Cull Off`。如果开启背面剔除，草片或树叶从背面观察时可能直接消失，VR 场景中的沉浸感会明显下降。对于森林这种视角自由移动的场景，双面渲染比普通静态截图更重要。

同时，修复后的 Shader 增加了 `ShadowCaster` pass。该 pass 使用同样的贴图 `alpha` 裁剪逻辑，使草、树叶等带透明区域的模型能够以正确轮廓参与阴影计算。否则，即使物体本身能够显示，也可能出现阴影缺失或阴影形状不合理的问题。

值得注意的是，本次透明裁剪使用贴图 `alpha`，而不是直接使用 `_Color.a`。这是因为原资源中部分树叶材质的颜色 `alpha` 可能为 0，如果直接按 `_Color.a` 裁剪，会把

树叶错误裁掉。改为依据贴图 alpha 判断，可以更符合资源原本的透明区域设计。

1.5.3 调整材质引用

Shader 修复完成后，还需要让相关材质引用新的兼容 Shader。实验中将以下材质统一改为引用 Fantasy Forest/StandardNoCulling:

- Assets/Fantasy Forest Environment Free Sample/Materials/dirt01.mat
- Assets/Fantasy Forest Environment Free Sample/Materials/grass01.mat
- Assets/Fantasy Forest Environment Free Sample/Materials/bark01_bottom.mat

另外,grassmesh.mat 和 tree_branches.mat 原本已经引用 Fantasy Forest/StandardNoCulling, 因此在 Shader 文件修复后会使用新的 URP 兼容实现。完成修改后, Unity 重新编译通过, Console 中没有 Shader 或脚本编译错误, 相关材质能够识别到 Shader: Fantasy Forest/StandardNoCulling。

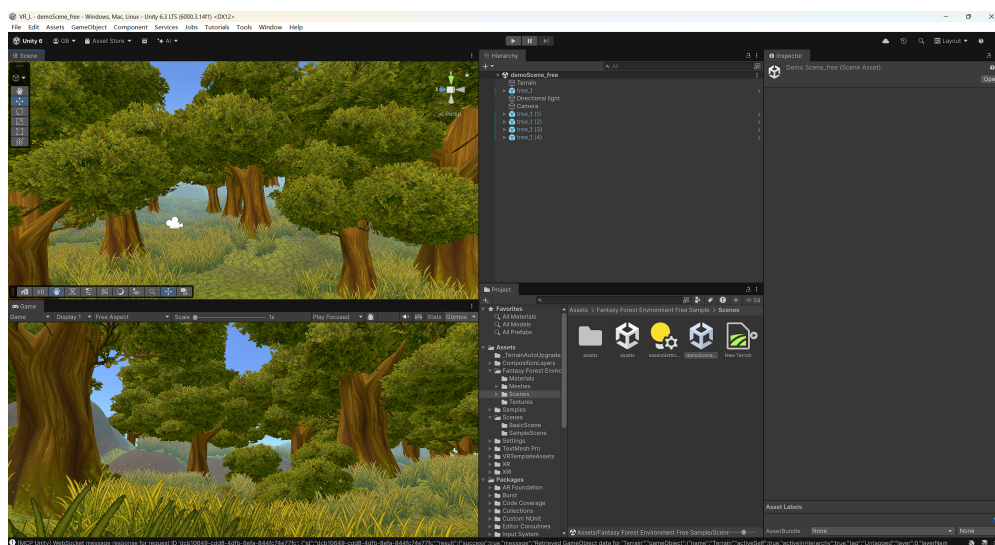


图 1.8 修复 Shader 和材质后森林场景正常显示

1.6 接入 VR 视角

1.6.1 复制 XR Origin

森林样例场景修复后，下一步是把 VR 模板项目中的 VR 视角接入森林场景。这里没有只复制单独的 Camera，而是复制完整的 XR Origin 预制体：

1 | Assets/VRTemplateAssets/Prefabs/Setup/Complete XR Origin Set Up Hands Variant.prefab

这样做的原因是，VR 视角并不只是一个摄像机。完整的 XR Origin 通常包含 Main Camera、Camera Offset、手部模型、控制器、输入行为和移动相关组件。如果只复制普通 Camera，场景可能可以显示画面，但无法完整保留 VR 模板中的手部、控制器和交互输入结构。

源场景为 `Assets/Scenes/SampleScene.unity`, 目标场景为 `Assets/Fantasy Forest Environment Free Sample/S` 实验在目标森林场景中新增 `XR Origin Hands (XR Rig)`, 并将其位置设置到原普通相机附近, 使用户进入 VR 后能够从合适的位置观察森林环境。

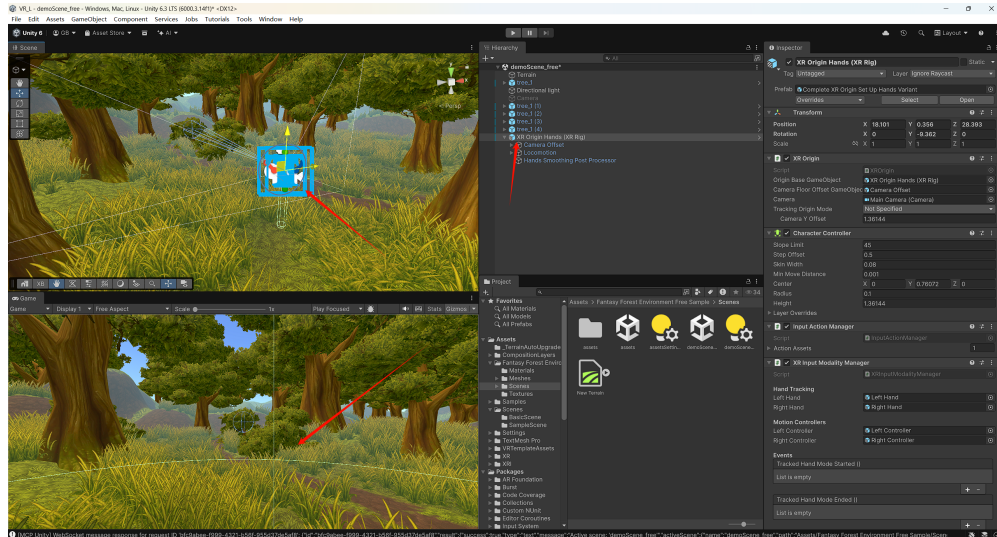


图 1.9 将 VR 模板中的 XR Origin 复制到森林场景

1.6.2 调整出生点与相机冲突

为了让进入场景后的观察位置合理, `XR Origin` 的位置参考森林场景原普通相机的位置设置:

- X: 17.5541019
- Y: 0
- Z: 28.302784

朝向参考原普通相机的水平朝向, 将 Y 轴旋转设置为 350.638153。这样进入 VR 场景后, 用户的初始视角能够面向森林环境主体, 而不是偏离地形或植被区域。

同时, 实验关闭了森林场景原来的普通 `Camera`。如果原相机和 `XR Rig` 内部的 `Main Camera` 同时存在并启用, 可能出现多个主相机同时渲染、`Audio Listener` 冲突或 VR 视角被普通相机抢占的问题。关闭原相机后, 场景由 `XR Origin` 内部的 `Main Camera` 负责显示, 更符合 VR 模板的运行方式。

1.7 运行展示与实验结果

完成 Shader 修复、材质引用调整和 `XR Origin` 接入后, 保存森林场景 `demoScene_free.unity`。验证结果表明, 场景中已经存在 `XR Origin Hands (XR Rig)`, 其内部包含 `Main Camera`、`Camera Offset`、手部、控制器、输入和移动相关组件; 原普通 `Camera` 已设置为 `inactive`; `Unity Console` 当前没有错误。

从画面效果看，修复后的森林场景能够正常显示地形、树木、草地和植被材质，不再出现大面积粉色材质。由于场景中的地形已经带有 `TerrainCollider`，VR 玩家移动和重力检测可以依赖地形碰撞，具备进一步扩展交互功能的基础。

本章实验最终完成了一个基于 Unity VR 模板的森林场景展示流程。实验过程说明，VR 场景搭建不仅包括导入模型和摆放资源，还包括项目模板选择、渲染管线确认、资源包兼容性分析、Shader 修改、材质修复、XR Origin 接入和相机冲突处理。尤其是旧资源包迁移到新版本 URP 项目时，粉色材质问题不能只从外观上处理，而应追溯到 Shader 与渲染管线的兼容关系。

1.8 本章小结

本章以森林为主题，在 Unity 中完成了 VR 场景的创建与展示。实验先使用 VR 模板创建项目，再导入 Fantasy Forest Environment Free Sample 资源包。由于资源包主要面向旧 Built-in Render Pipeline，而当前项目使用 Unity 6 + URP 17.3.0，导入后出现大面积粉色材质。通过将资源包自带的 `StandardNoCulling.shader` 改写为 URP HLSL Shader，并调整相关材质引用，草地、树叶、树干和地面等资源恢复正常显示。

在场景交互方面，实验将 VR 模板中的完整 XR Origin 复制到森林样例场景，而不是只复制普通相机，从而保留 VR 相机、手部、控制器、输入和移动相关组件。最终，森林场景能够以 VR 视角进入和展示，为后续添加交互任务、路径漫游或识别结果可视化提供了基础。

2 结论

本文围绕虚拟现实课程中的视觉分析与三维场景展示任务，完成了浏览器端人脸分析实验、dlib 人脸描述子训练示例分析，以及 Unity 森林 VR 场景构建三个部分的工作。报告首先分析了基于 `face-api.js` 的浏览器端人脸分析项目，说明了静态页面如何加载预训练模型，并完成表情识别、年龄性别估计和摄像头实时人脸追踪。随后，报告进一步分析 dlib 官方度量学习示例，从训练数据组织、mini-batch 采样、ResNet 描述子网络、损失函数和训练后阈值验证等方面解释了人脸描述子为什么可以通过距离进行比较。

在虚拟现实场景构建方面，本文使用 Unity VR 模板创建项目，并以森林为主题导入 Fantasy Forest Environment Free Sample 资源包。实验过程中遇到旧 Built-in 渲染管线资源导入 Unity 6 + URP 后材质变粉的问题。通过分析 Shader 与渲染管线的兼容关系，本文将资源包自带的 `StandardNoCulling.shader` 改写为 URP 可用的 HLSL Shader，并调整相关材质引用，使森林地形、草地、树叶和树干能够正常显示。最后，通过复制完整 XR Origin 到森林场景，完成了 VR 视角接入和运行展示。

通过这些实验可以看出，视觉智能和虚拟现实应用都不是单一算法或单一界面能够完成的任务。浏览器端人脸分析需要把模型权重、前端页面、媒体输入和 Canvas 可视化连接起来；dlib 度量学习示例需要理解数据组织、网络结构、损失函数和阈值验证之间的关系；Unity VR 场景则需要同时处理资源导入、渲染管线、Shader、材质、相机和 XR 组件。三部分内容虽然侧重点不同，但共同体现了课程实验中“模型能力、工程实现和展示环境”之间的联系。

本文仍存在一些不足。人脸分析部分主要使用预训练模型，没有重新训练和系统评估模型性能；dlib 示例分析以代码理解和运行记录为主，没有使用大规模数据集完成完整训练；Unity VR 场景已经完成基本展示，但交互内容仍较少，尚未加入任务引导、物体拾取、路径漫游或与人脸识别结果联动的展示模块。后续可以在现有森林 VR 场景基础上加入更多交互元素，并尝试把前两章的人脸识别结果以虚拟展板、提示信息或场景触发事件的形式呈现在 VR 环境中，使视觉识别与虚拟现实展示形成更完整的应用闭环。

参考文献

- [1] Vincent Muhler. face-api.js: Javascript api for face detection and face recognition in the browser and node.js with tensorflow.js. GitHub repository, 2018.
- [2] Vincent Muhler. face-api.js model weights. GitHub repository.
- [3] Davis E. King. Dlib-ml: A machine learning toolkit. *Journal of Machine Learning Research*, 2009.
- [4] Davis E. King. dlib c++ library. GitHub repository.
- [5] World Wide Web Consortium. Media capture and streams. W3C Recommendation.
- [6] Bootstrap Team. Bootstrap documentation.

附录 人脸分析实验关键代码

本附录列出浏览器端人脸分析实验中与功能实现直接相关的页面结构和业务脚本片段。为避免重复展示第三方库源码，附录不收录 `face-api.min.js`，只展示页面如何引入模型库、承载输入输出区域，以及三个实验模块如何完成模型加载、图片推理、结果渲染和摄像头实时追踪。

页面入口与展示容器

图片表情识别页面在文档头部延迟加载 `face-api.min.js` 和当前页面的业务脚本。主体区域左侧提供图片上传控件和识别结果容器，右侧的 `analysis-stage` 用于插入用户上传的图片和覆盖绘制结果的画布。

这里保留脚本标签的原因是，代码引入顺序直接影响页面能否运行。`face-api.min.js` 必须先于业务脚本加载，业务脚本才能访问 `face-api` 对象；`defer` 则保证脚本在 DOM 解析完成后按顺序执行，使业务脚本能够拿到上传控件、展示区域等真实节点。

```
21 <script defer src="dist/face-api.min.js"></script>
22 <script defer src="dist/expression_script.js"></script>

75 <section class="panel control-panel">
76   <h2 class="panel-heading"> 上传图片 </h2>
77   <input id="upload" type="file" accept="image/*" />
78   <label for="upload" class="upload-dropzone">
79     <span class="upload-label"> 选择图片 </span>
80     <span class="upload-hint"> 建议使用清晰、正脸较明显的照片。</span>
81   </label>
82   <p class="panel-note"> 支持常见 JPG、PNG 图片格式。</p>
83 </section>

84 <section class="panel info-panel">
85   <h2 class="panel-heading"> 识别结果 </h2>
86   <div id="expression-results" class="result-summary">
87     <div class="result-empty">
88       <strong> 等待识别 </strong>
89       <span> 上传图片后，这里会同步显示表情识别结果副本。</span>
90     </div>
91   </div>
92 </section>
93 </aside>

94 <section class="panel result-panel">
95   <div id="analysis-stage" class="analysis-stage">
96     <div class="stage-empty" data-empty-state>
97       <strong> 等待图片 </strong>
98       <span> 上传后在这里显示表情识别结果。</span>
99     </div>
```

年龄与性别估计页面的入口结构与表情识别页面基本一致，只是业务脚本替换为 `dist/estimation_script.js`，结果容器也变为 `estimation-results`。这种“页面结构相同、业务脚本不同”的组织方式有利于复用样式和交互习惯。

```

21 <script defer src="dist/face-api.min.js"></script>
22 <script defer src="dist/estimation_script.js"></script>

75 <section class="panel control-panel">
76 <h2 class="panel-heading"> 上传图片 </h2>
77 <input id="upload" type="file" accept="image/*" />
78 <label for="upload" class="upload-dropzone">
79 <span class="upload-label"> 选择图片 </span>
80 <span class="upload-hint"> 建议使用清晰、无遮挡的人像照片。</span>
81 </label>
82 <p class="panel-note"> 支持常见 JPG、PNG 图片格式。</p>
83 </section>

84 <section class="panel info-panel">
85 <h2 class="panel-heading"> 识别结果 </h2>
86 <div id="estimation-results" class="result-summary">
87 <div class="result-empty">
88 <strong> 等待识别 </strong>
89 <span> 上传图片后，这里会同步显示年龄与性别估计结果。</span>
90 </div>
91 </div>
92 </section>
93 </aside>

94 <section class="panel result-panel">
95 <div id="analysis-stage" class="analysis-stage">
96 <div class="stage-empty" data-empty-state>
97 <strong> 等待图片 </strong>
98 <span> 上传后在这里显示年龄与性别估计结果。</span>

```

摄像头实时追踪页面同样通过脚本标签加载模型库和业务逻辑。页面中的追踪区域 `tracking-stage` 用来叠加视频与检测画布。

`video` 元素负责承接浏览器摄像头视频流，并为后续逐帧识别提供输入。

```

21 <script defer src="dist/face-api.min.js"></script>
22 <script defer src="dist/webcam_script.js"></script>

101 <section class="panel tracking-panel">
102 <div id="tracking-stage" class="tracking-stage">
103 <div class="stage-empty" data-empty-state>
104 <strong> 等待摄像头权限 </strong>
105 <span> 授权后在这里显示实时画面与识别结果。</span>
106 </div>
107 <video
108   id="video"
109   width="1280"
110   height="720"
111   autoplay
112   muted
113   playsinline
114 ></video>
115 </div>

```

图片表情识别脚本

表情识别脚本首先取得页面中的上传控件、展示区域和结果区域，并定义本地模型路径。随后通过 并行加载检测、关键点、人脸描述子、表情识别和 SSD MobileNetV1

模型，全部模型加载完成后才进入 流程。

```
1  const imageUpload = document.getElementById("upload");
2  const analysisStage = document.getElementById("analysis-stage");
3  const expressionResults = document.getElementById("expression-results");
4  const MODEL_URL = "./models";

5  const EXPRESSION_LABELS = {
6    neutral: "中性",
7    happy: "高兴",
8    sad: "难过",
9    angry: "生气",
10   fearful: "害怕",
11   disgusted: "厌恶",
12   surprised: "惊讶",
13 };
14 const EXPRESSION_ORDER = Object.keys(EXPRESSION_LABELS);

15 Promise.all([
16   faceapi.nets.tinyFaceDetector.loadFromUri(MODEL_URL),
17   faceapi.nets.faceLandmark68Net.loadFromUri(MODEL_URL),
18   faceapi.nets.faceRecognitionNet.loadFromUri(MODEL_URL),
19   faceapi.nets.faceExpressionNet.loadFromUri(MODEL_URL),
20   faceapi.nets.ssdMobilenetv1.loadFromUri(MODEL_URL),
21 ]).then(start);
```

这一段代码体现了两个关键点。第一，指向 `./models`，说明权重从项目本地目录加载，不依赖远程网络。第二，模型加载使用 `Promise.all`，所有异步加载任务都完成后才调用 `start`。如果缺少这一步等待，用户上传图片时模型可能尚未准备好，推理调用会失败。

为了让模型输出更容易阅读，脚本将 `face-api.js` 返回的英文表情标签映射为中文标签，并按检测框横坐标排序多张人脸。结果渲染函数会选出概率最高的主表情，同时把七类表情概率渲染为进度条，便于观察模型对不同表情类别的置信度分布。

```
38 function getSortedDetections(detections) {
39   return [...detections].sort(
40     (first, second) => first.detection.box.x - second.detection.box.x
41   );
42 }

43 function getTopExpression(expressions) {
44   return EXPRESSION_ORDER.reduce(
45     (currentTop, key) => {
46       const score = expressions[key] || 0;
47       return score > currentTop.score ? { key, score } : currentTop;
48     },
49     { key: EXPRESSION_ORDER[0], score: expressions[EXPRESSION_ORDER[0]] || 0 }
50   );
51 }

52 function renderExpressionResults(detections) {
53   const sortedDetections = getSortedDetections(detections);

54   if (!sortedDetections.length) {
55     renderEmptyState(
56       expressionResults,
```

```

57     " 未检测到人脸",
58     " 请尝试上传清晰、正脸更明显的照片。"
59 );
60     return;
61 }

62 expressionResults.innerHTML = sortedDetections
63     .map((detection, index) => {
64         const topExpression = getTopExpression(detection.expressions);
65         const metrics = EXPRESSION_ORDER.map((key) => {
66             const score = detection.expressions[key] || 0;

67             return `
68                 <li class="result-metric">
69                     <div class="result-metric__row">
70                         <span class="result-metric__label">${EXPRESSION_LABELS[key]}</span>
71                         <span class="result-metric__value">${formatPercent(score)}</span>
72                     </div>
73                     <div class="result-bar"><span style="width: ${Math.round(
74                         score * 100
75                     )}%></span></div>
76                 </li>
77             `;
78         }).join("");

79         return `
80             <article class="result-card">
81                 <div class="result-card__header">
82                     <span class="result-card__title"> 人脸 ${index + 1}</span>
83                     <span class="status-pill">${EXPRESSION_LABELS[topExpression.key]}</span>
84                 </div>
85                 <p class="result-card__highlight">
86                     主表情: <strong>${EXPRESSION_LABELS[topExpression.key]}</strong>
87                     (${formatPercent(topExpression.score)})
88                 </p>
89                 <ul class="result-list">${metrics}</ul>
90             </article>
91         `;
92     }).join("");
93 }
94 }

```

图片上传监听器是该模块的核心入口。用户选择文件后，脚本先清理旧图片和旧画布，再将文件转换为浏览器图片对象，创建覆盖画布并匹配尺寸。随后，脚本依次执行人脸检测、关键点定位、人脸描述子提取和表情识别，最后将结果缩放到展示尺寸并绘制检测框和表情标签。

```

102 async function start() {
103     const container = document.createElement("div");
104     container.className = "analysis-stage__media";
105     analysisStage.append(container);

106     let image;
107     let canvas;

108     imageUpload.addEventListener("change", async () => {
109         if (!imageUpload.files.length) {

```



```
110     return;
111   }

112   renderEmptyState(expressionResults, " 正在识别", " 图片分析中, 请稍候。");

113   if (image) {
114     image.remove();
115   }
116   if (canvas) {
117     canvas.remove();
118   }

119   image = await faceapi.bufferToImage(imageUpload.files[0]);
120   container.append(image);

121   canvas = faceapi.createCanvasFromMedia(image);
122   container.append(canvas);

123   const displaySize = { width: image.width, height: image.height };
124   faceapi.matchDimensions(canvas, displaySize);

125   const detections = await faceapi
126     .detectAllFaces(image)
127     .withFaceLandmarks()
128     .withFaceDescriptors()
129     .withFaceExpressions();

130   const resizedDetections = faceapi.resizeResults(detections, displaySize);
131   faceapi.draw.drawDetections(canvas, resizedDetections);
132   faceapi.draw.drawFaceExpressions(canvas, resizedDetections);
133   renderExpressionResults(resizedDetections);

134   analysisStage.classList.add("stage-ready");
135   });
136 }
```

其中 负责把本地文件对象转换成浏览器可绘制的图片元素，创建与图片对应的覆盖画布，保证画布坐标系与图片显示尺寸一致。链式调用 表示先检测所有人脸，再补充关键点、人脸描述子和表情结果。最后的 用于把模型输出坐标映射回展示尺寸。

年龄与性别估计脚本

年龄与性别估计脚本与表情识别脚本结构相近，但加载的任务模型不同。它除了检测、关键点和描述子模型外，还加载，用于输出估计年龄、性别标签和性别分类置信度。

```
1  const imageUpload = document.getElementById("upload");
2  const analysisStage = document.getElementById("analysis-stage");
3  const estimationResults = document.getElementById("estimation-results");
4  const MODEL_URL = "./models";

5  const GENDER_LABELS = {
6    male: " 男性",
```

```

7   female: " 女性",
8   };

9   Promise.all([
10    faceapi.nets.tinyFaceDetector.loadFromUri(MODEL_URL),
11    faceapi.nets.faceLandmark68Net.loadFromUri(MODEL_URL),
12    faceapi.nets.faceRecognitionNet.loadFromUri(MODEL_URL),
13    faceapi.nets.ageGenderNet.loadFromUri(MODEL_URL),
14    faceapi.nets.ssdMobilenetv1.loadFromUri(MODEL_URL),
15  ]).then(start);

```

这一模块同样使用本地 `models` 目录。与表情识别相比，它把任务网络从 `faceapi` 换成 `faceapi`。这说明页面的基础流程可以复用：只要输入仍然是人脸区域，后续可以在同一检测结果上接入不同属性估计网络。

结果渲染部分把模型返回的 `estimationResults` 和 `detections` 映射为中文展示文本，并将年龄估计值四舍五入后写入结果卡片。性别置信度继续用百分比和进度条表示，使侧边栏结果能够与右侧检测框形成对照。

```

38 function renderEstimationResults(detections) {
39   const sortedDetections = getSortedDetections(detections);

40   if (!sortedDetections.length) {
41     renderEmptyState(
42       estimationResults,
43       " 未检测到人脸",
44       " 请尝试上传清晰、无遮挡的人像照片。"
45     );
46     return;
47   }

48   estimationResults.innerHTML = sortedDetections
49     .map((detection, index) => {
50     const gender = detection.gender || "male";
51     const genderLabel = GENDER_LABELS[gender] || gender;
52     const confidence = detection.genderProbability || 0;

53     return `
54       <article class="result-card">
55         <div class="result-card__header">
56           <span class="result-card__title"> 人脸 ${index + 1}</span>
57           <span class="status-pill">${genderLabel}</span>
58         </div>
59         <div class="result-detail-grid">
60           <div class="result-detail">
61             <span class="result-detail__label"> 年龄估计 </span>
62             <span class="result-detail__value">${Math.round(
63               detection.age
64             )} 岁 </span>
65           </div>
66           <div class="result-detail">
67             <span class="result-detail__label"> 性别判断 </span>
68             <span class="result-detail__value">${genderLabel}</span>
69           </div>
70         </div>
71       <ul class="result-list">

```

```

72         <li class="result-metric">
73             <div class="result-metric__row">
74                 <span class="result-metric__label"> 性别置信度 </span>
75                 <span class="result-metric__value">${formatPercent(
76                     confidence
77                 )}</span>
78             </div>
79             <div class="result-bar"><span style="width: ${Math.round(
80                 confidence * 100
81             )}%"></span></div>
82         </li>
83     </ul>
84 </article>
85 `;
86 })
87 .join("");
88 }

```

图片推理流程仍然从上传事件开始。与表情识别不同的是，链式调用最后使用 `renderEmptyState`，并在画布上额外通过 `faceapi.matchDimensions` 绘制包含年龄和性别的检测框标签。

```

93 async function start() {
94     const container = document.createElement("div");
95     container.className = "analysis-stage__media";
96     analysisStage.append(container);
97
98     let image;
99     let canvas;
100
101     imageUpload.addEventListener("change", async () => {
102         if (!imageUpload.files.length) {
103             return;
104         }
105
106         renderEmptyState(estimationResults, " 正在识别", " 图片分析中，请稍候。");
107
108         if (image) {
109             image.remove();
110         }
111         if (canvas) {
112             canvas.remove();
113         }
114
115         image = await faceapi.bufferToImage(imageUpload.files[0]);
116         container.append(image);
117
118         canvas = faceapi.createCanvasFromMedia(image);
119         container.append(canvas);
120
121         const displaySize = { width: image.width, height: image.height };
122         faceapi.matchDimensions(canvas, displaySize);
123
124         const detections = await faceapi
125             .detectAllFaces(image)
126             .withFaceLandmarks()
127             .withFaceDescriptors()
128             .withAgeAndGender();
129
130         const resizedDetections = faceapi.resizeResults(detections, displaySize);

```

```
122     faceapi.draw.drawDetections(canvas, resizedDetections);
123
124     resizedDetections.forEach((detection, index) => {
125         const box = resizedDetections[index].detection.box;
126         const drawBox = new faceapi.draw.DrawBox(box, {
127             label: `${Math.round(detection.age)} years old ${detection.gender}`,
128         });
129         drawBox.draw(canvas);
130     });
131
132     renderEstimationResults(resizedDetections);
133
134     analysisStage.classList.add("stage-ready");
135 }
136 }
```

从代码结构看，年龄与性别估计和表情识别最大的差异只在两处：一是模型加载列表不同，二是链式推理末端不同。其余的文件读取、图片插入、画布创建、尺寸匹配、检测结果缩放等流程完全相似。这也说明前端人脸分析页面适合按“基础检测流程 + 任务网络”的方式扩展。

摄像头实时追踪脚本

摄像头脚本在页面初始化时保存视频元素、追踪区域、状态区和结果区引用，并维护覆盖画布、定时器和结果刷新时间等状态。模型加载完成后进入，为后续实时识别做准备。

```
1  const video = document.getElementById("video");
2  const trackingStage = document.getElementById("tracking-stage");
3  const webcamStatus = document.getElementById("webcam-status");
4  const webcamResults = document.getElementById("webcam-results");
5  const MODEL_URL = "./models";
6  let canvas;
7  let detectionInterval;
8  let lastResultsMarkup = "";
9  let lastResultsRenderAt = 0;
10
11  const EXPRESSION_LABELS = {
12      neutral: "中性",
13      happy: "高兴",
14      sad: "难过",
15      angry: "生气",
16      fearful: "害怕",
17      disgusted: "厌恶",
18      surprised: "惊讶",
19  };
20
21  const EXPRESSION_ORDER = Object.keys(EXPRESSION_LABELS);
22  const SIDEBAR_REFRESH_MS = 280;
23
24  Promise.all([
25      faceapi.nets.tinyFaceDetector.loadFromUri(MODEL_URL),
26      faceapi.nets.faceLandmark68Net.loadFromUri(MODEL_URL),
27      faceapi.nets.faceRecognitionNet.loadFromUri(MODEL_URL),
28      faceapi.nets.faceExpressionNet.loadFromUri(MODEL_URL),
29  ])
```

```
26 | }).then(startVideo);
27 |
28 | function getDisplaySize() {
29 |     return {
30 |         width: Math.max(trackingStage.clientWidth, 1),
31 |         height: Math.max(trackingStage.clientHeight, 1),
32 |     };
33 | }
34 |
35 | function syncCanvasSize() {
36 |     if (!canvas) {
37 |         return getDisplaySize();
38 |     }
39 |
40 |     const displaySize = getDisplaySize();
41 |
42 |     if (canvas.width !== displaySize.width || canvas.height !== displaySize.height) {
43 |         faceapi.matchDimensions(canvas, displaySize);
44 |     }
45 |
46 |     return displaySize;
47 | }
```

与图片页面不同，摄像头页面需要维护跨帧状态。保存覆盖画布，保存定时器编号，和 用于控制侧边栏刷新频率。这些变量保证页面在连续视频输入下不会不断创建新画布或过度更新 DOM。

实时页面需要频繁更新侧边栏，因此脚本把结果 HTML 的生成和渲染拆开处理。渲染函数会比较本次结果与上次结果，并使用 控制刷新闻隔，避免每 100 ms 都重复写入 DOM。

```
87 | function getTopExpression(expressions) {
88 |     return EXPRESSION_ORDER.reduce(
89 |         (currentTop, key) => {
90 |             const score = expressions[key] || 0;
91 |             return score > currentTop.score ? { key, score } : currentTop;
92 |         },
93 |         { key: EXPRESSION_ORDER[0], score: expressions[EXPRESSION_ORDER[0]] || 0 }
94 |     );
95 | }
96 |
97 | function buildResultsMarkup(detections) {
98 |     const sortedDetections = getSortedDetections(detections);
99 |
100 |     if (!sortedDetections.length) {
101 |         return `
102 |             <div class="result-empty">
103 |                 <strong> 未检测到人脸 </strong>
104 |                 <span> 当前画面中还没有检测到清晰人脸。</span>
105 |             </div>
106 |         `;
107 |     }
108 |
109 |     return sortedDetections
110 |         .map((detection, index) => {
111 |             const topExpression = getTopExpression(detection.expressions);
```

```

109     const metrics = EXPRESSION_ORDER.map((key) => {
110         const score = detection.expressions[key] || 0;

111         return `
112             <li class="result-metric">
113                 <div class="result-metric__row">
114                     <span class="result-metric__label">${EXPRESSION_LABELS[key]}</span>
115                     <span class="result-metric__value">${formatPercent(score)}</span>
116                 </div>
117                 <div class="result-bar"><span style="width: ${Math.round(
118                     score * 100
119                 )}%></span></div>
120             </li>
121         `;
122     }).join("");

123     return `
124         <article class="result-card">
125             <div class="result-card__header">
126                 <span class="result-card__title">人脸 ${index + 1}</span>
127                 <span class="status-pill">${EXPRESSION_LABELS[topExpression.key]}</span>
128             </div>
129             <p class="result-card__highlight">
130                 主表情: <strong>${EXPRESSION_LABELS[topExpression.key]}</strong>
131                 (${formatPercent(topExpression.score)})
132             </p>
133             <ul class="result-list">${metrics}</ul>
134         </article>
135     `;
136     })
137     .join("");
138 }

139 function renderWebcamResults(detections, force = false) {
140     const markup = buildResultsMarkup(detections);
141     const now = Date.now();

142     if (!force && markup === lastResultsMarkup) {
143         return;
144     }

145     if (!force && now - lastResultsRenderAt < SIDEBAR_REFRESH_MS) {
146         return;
147     }

148     webcamResults.innerHTML = markup;
149     lastResultsMarkup = markup;
150     lastResultsRenderAt = now;
151 }

```

摄像头启动函数兼容了现代 和旧版浏览器前缀接口。脚本会先显示等待授权状态，在用户允许访问摄像头后把视频流写入；如果浏览器不支持或授权失败，则更新状态区和结果区提示。

```

162 function startVideo() {
163     renderStatus(" 等待授权", " 未连接", " 允许浏览器访问摄像头后开始实时识别。");
164     renderEmptyResults(
165         " 等待识别",

```



```

166     " 摄像头开始工作后，这里会同步显示表情识别结果副本。",
167     true
168 );

169 const legacyGetUserMedia =
170     navigator.getUserMedia ||
171     navigator.webkitGetUserMedia ||
172     navigator.mozGetUserMedia;

173 const getUserMedia =
174     navigator.mediaDevices && navigator.mediaDevices.getUserMedia
175     ? (constraints) => navigator.mediaDevices.getUserMedia(constraints)
176     : legacyGetUserMedia
177     ? (constraints) =>
178         new Promise((resolve, reject) => {
179             legacyGetUserMedia.call(navigator, constraints, resolve, reject);
180         })
181     : null;

182 if (!getUserMedia) {
183     console.error("Camera API is not supported in this browser.");
184     renderStatus(" 摄像头不可用", " 不支持", " 当前浏览器不支持摄像头访问接口。");
185     renderEmptyResults(
186         " 无法开始识别",
187         " 请更换支持摄像头访问的浏览器后重试。",
188         true
189     );
190     return;
191 }

192 renderStatus(" 正在连接", " 连接中", " 正在请求摄像头权限，请稍候。");

193 getUserMedia({ video: {} })
194     .then((stream) => {
195         video.srcObject = stream;
196         renderStatus(" 摄像头已连接", " 已连接", " 画面已连接，等待浏览器开始播放视频流。");
197     })
198     .catch((error) => {
199         console.error(error);
200         renderStatus(
201             " 访问失败",
202             " 不可用",
203             " 未能获取摄像头权限，或者当前设备不可用。"
204         );
205         renderEmptyResults(
206             " 无法开始识别",
207             " 请检查浏览器授权和摄像头设备后重试。",
208             true
209         );
210     });
211 }

```

当视频开始播放时，脚本创建覆盖画布并启动定时器。每轮定时任务都会同步画布尺寸，对当前视频帧执行人脸检测、关键点定位和表情识别，再清空上一帧标注并重绘最新检测框、关键点和表情标签。视频暂停或结束时，脚本及时清除定时器，避免后台继续执行无意义的推理。

```

218 video.addEventListener("play", () => {

```

```

219     if (!canvas) {
220         canvas = faceapi.createCanvasFromMedia(video);
221         trackingStage.append(canvas);
222     }

223     syncCanvasSize();
224     trackingStage.classList.add("stage-live");
225     renderStatus(" 正在识别", " 识别中", " 已连接摄像头, 正在同步更新实时识别结果。");

226     window.clearInterval(detectionInterval);
227     detectionInterval = window.setInterval(async () => {
228         const displaySize = syncCanvasSize();
229         const detections = await faceapi
230             .detectAllFaces(video, new faceapi.TinyFaceDetectorOptions())
231             .withFaceLandmarks()
232             .withFaceExpressions();

233         const resizedDetections = faceapi.resizeResults(detections, displaySize);
234         canvas.getContext("2d").clearRect(0, 0, canvas.width, canvas.height);
235         faceapi.draw.drawDetections(canvas, resizedDetections);
236         faceapi.draw.drawFaceLandmarks(canvas, resizedDetections);
237         faceapi.draw.drawFaceExpressions(canvas, resizedDetections);
238         renderWebcamResults(resizedDetections);
239     }, 100);
240 });

241 window.addEventListener("resize", () => {
242     syncCanvasSize();
243 });

244 ["pause", "ended"].forEach((eventName) => {
245     video.addEventListener(eventName, () => {
246         window.clearInterval(detectionInterval);
247         renderStatus(" 识别已暂停", " 已暂停", " 摄像头画面已暂停, 左侧结果已停止更新。");
248         renderEmptyResults(
249             " 识别已暂停",
250             " 恢复摄像头画面后, 这里会重新同步显示识别结果。",
251             true
252         );
253     });
254 });

```

这段代码中的 `faceapi.TinyFaceDetectorOptions` 表明实时检测采用轻量级检测器。每 100 ms 执行一次推理大致相当于每秒最多 10 次检测, 实际帧率还会受到设备性能、浏览器和摄像头分辨率影响。定时器中先调用 `clearRect` 清除上一帧绘制内容, 再绘制新一帧结果, 否则旧检测框会残留在画布上。

开源人脸识别模型训练代码

本项目网页端直接使用 `face-api.js` 的预训练权重, 并没有在浏览器项目中重新训练深度网络。为了说明人脸识别模型的训练思想, 本节引用已经克隆到 `code/dlib/` 目录中的 `dlib` 开源训练示例 `dnn_metric_learning_on_images_ex.cpp`。该示例展示了如何使用 `dlib` 训练人脸嵌入模型, 使同一身份的人脸向量距离更近, 不同身份的人脸向量距离

更远。

示例开头说明了深度度量学习在人脸识别中的作用：模型不直接把图像分类为固定身份，而是学习一个向量空间，再用距离或最近邻方法完成识别。

```
1 // The contents of this file are in the public domain. See
  ↳ LICENSE_FOR_EXAMPLE_PROGRAMS.txt
2 /*
3     This is an example illustrating the use of the deep learning tools from the
4     dlib C++ Library. In it, we will show how to use the loss_metric layer to do
5     metric learning on images.
6
7     The main reason you might want to use this kind of algorithm is because you
8     would like to use a k-nearest neighbor classifier or similar algorithm, but
9     you don't know a good way to calculate the distance between two things. A
10    popular example would be face recognition. There are a whole lot of papers
11    that train some kind of deep metric learning algorithm that embeds face
12    images in some vector space where images of the same person are close to each
13    other and images of different people are far apart. Then in that vector
14    space it's very easy to do face recognition with some kind of k-nearest
15    neighbor classifier.
16
17    In this example we will use a version of the ResNet network from the
18    dnn_imagenet_ex.cpp example to learn to map images into some vector space where
19    pictures of the same person are close and pictures of different people are far
20    apart.
21
22    You might want to read the simpler introduction to the deep metric learning
23    API, dnn_metric_learning_ex.cpp, before reading this example. You should
24    also have read the examples that introduce the dlib DNN API before
25    continuing. These are dnn_introduction_ex.cpp and dnn_introduction2_ex.cpp.
26
27    */
```

训练数据按身份目录组织。顶层目录下的每个子目录表示一个人物身份，子目录内保存该人物的多张图片。加载函数遍历这些子目录，形成按身份分组的图片列表，为后续 mini-batch 采样做准备。

```
38 // We will need to create some functions for loading data. This program will
39 // expect to be given a directory structured as follows:
40 //     top_level_directory/
41 //         person1/
42 //             image1.jpg
43 //             image2.jpg
44 //             image3.jpg
45 //         person2/
46 //             image4.jpg
47 //             image5.jpg
48 //             image6.jpg
49 //         person3/
50 //             image7.jpg
51 //             image8.jpg
52 //             image9.jpg
53 //
54 // The specific folder and image names don't matter, nor does the number of folders or
55 // images. What does matter is that there is a top level folder, which contains
56 // subfolders, and each subfolder contains images of a single person.
```

```

57 // This function spiders the top level directory and obtains a list of all the
58 // image files.
59 std::vector<std::vector<string>> load_objects_list (
60     const string& dir
61 )
62 {
63     std::vector<std::vector<string>> objects;
64     for (auto subdir : directory(dir).get_dirs())
65     {
66         std::vector<string> imgs;
67         for (auto img : subdir.get_files())
68             imgs.push_back(img);
69
70         if (imgs.size() != 0)
71             objects.push_back(imgs);
72     }
73     return objects;
74 }

```

mini-batch 采样函数继续从每个身份下随机选择图片，并生成与图片对应的整数标签。这个标签不是人物姓名，而是身份在外层向量中的索引。函数还进行了颜色扰动和图像抖动，并检查同一 batch 中图片尺寸是否一致。

```

77 // This function takes the output of load_objects_list() as input and randomly
78 // selects images for training. It should also be pointed out that it's really
79 // important that each mini-batch contain multiple images of each person. This
80 // is because the metric learning algorithm needs to consider pairs of images
81 // that should be close (i.e. images of the same person) as well as pairs of
82 // images that should be far apart (i.e. images of different people) during each
83 // training step.
84 void load_mini_batch (
85     const size_t num_people, // how many different people to include
86     const size_t samples_per_id, // how many images per person to select.
87     dlib::rand& rnd,
88     const std::vector<std::vector<string>>& objs,
89     std::vector<matrix<rgb_pixel>>& images,
90     std::vector<unsigned long>& labels
91 )
92 {
93     images.clear();
94     labels.clear();
95     DLIB_CASSERT(num_people <= objs.size(), "The dataset doesn't have that many people
96     ↳ in it.");
97
98     std::vector<bool> already_selected(objs.size(), false);
99     matrix<rgb_pixel> image;
100     for (size_t i = 0; i < num_people; ++i)
101     {
102         size_t id = rnd.get_random_32bit_number()%objs.size();
103         // don't pick a person we already added to the mini-batch
104         while(already_selected[id])
105             id = rnd.get_random_32bit_number()%objs.size();
106         already_selected[id] = true;
107
108         for (size_t j = 0; j < samples_per_id; ++j)
109         {
110             const auto& obj = objs[id][rnd.get_random_32bit_number()%objs[id].size()];

```

```

108         load_image(image, obj);
109         images.push_back(std::move(image));
110         labels.push_back(id);
111     }
112 }
113
114 // You might want to do some data augmentation at this point. Here we do some
115 // simple
116 // color augmentation.
117 for (auto&& crop : images)
118 {
119     disturb_colors(crop,rnd);
120     // Jitter most crops
121     if (rnd.get_random_double() > 0.1)
122         crop = jitter_image(crop,rnd);
123 }
124
125 // All the images going into a mini-batch have to be the same size. And really,
126 // all
127 // the images in your entire training dataset should be the same size for what we
128 // are
129 // doing to make the most sense.
130 DLIB_CASSERT(images.size() > 0);
131 for (auto&& img : images)
132 {
133     DLIB_CASSERT(img.nr() == images[0].nr() && img.nc() == images[0].nc(),
134         "All the images in a single mini-batch must be the same size.");
135 }
136 }

```

网络结构部分定义了训练网络和测试网络。训练网络使用带批归一化的 ResNet 主体，输出层为，损失层为；测试网络则把批归一化替换为固定仿射变换，便于推理阶段稳定使用。

```

144 template <template <int,template<typename>class,int,typename> class block, int N,
145     <template<typename>class BN, typename SUBNET>
146 using residual = add_prev1<block<N,BN,1,tag1<SUBNET>>>>;
147
148 template <template <int,template<typename>class,int,typename> class block, int N,
149     <template<typename>class BN, typename SUBNET>
150 using residual_down =
151     <add_prev2<avg_pool<2,2,2,2,skip1<tag2<block<N,BN,2,tag1<SUBNET>>>>>>>>>;
152
153 template <int N, template <typename> class BN, int stride, typename SUBNET>
154 using block = BN<con<N,3,3,1,1,relu<BN<con<N,3,3,stride,stride,SUBNET>>>>>>>>;
155
156 template <int N, typename SUBNET> using res =
157     <relu<residual<block<N,bn_con,SUBNET>>>>;
158 template <int N, typename SUBNET> using ares =
159     <relu<residual<block<N,affine,SUBNET>>>>;
160 template <int N, typename SUBNET> using res_down =
161     <relu<residual_down<block<N,bn_con,SUBNET>>>>;
162 template <int N, typename SUBNET> using ares_down =
163     <relu<residual_down<block<N,affine,SUBNET>>>>;
164
165 // -----
166 <-----

```

```

155 template <typename SUBNET> using level0 = res_down<256,SUBNET>;
156 template <typename SUBNET> using level1 = res<256,res<256,res_down<256,SUBNET>>>;
157 template <typename SUBNET> using level2 = res<128,res<128,res_down<128,SUBNET>>>;
158 template <typename SUBNET> using level3 = res<64,res<64,res<64,res_down<64,SUBNET>>>>;
159 template <typename SUBNET> using level4 = res<32,res<32,res<32,SUBNET>>>;

160 template <typename SUBNET> using alevel0 = ares_down<256,SUBNET>;
161 template <typename SUBNET> using alevel1 = ares<256,ares<256,ares_down<256,SUBNET>>>;
162 template <typename SUBNET> using alevel2 = ares<128,ares<128,ares_down<128,SUBNET>>>;
163 template <typename SUBNET> using alevel3 =
↪ ares<64,ares<64,ares<64,ares_down<64,SUBNET>>>>;
164 template <typename SUBNET> using alevel4 = ares<32,ares<32,ares<32,SUBNET>>>;

165 // training network type
166 using net_type = loss_metric<fc_no_bias<128,avg_pool_everything<
167     level0<
168     level1<
169     level2<
170     level3<
171     level4<
172     max_pool<3,3,2,2,relu<bn_con<con<32,7,7,2,2,
173     input_rgb_image
174     >>>>>>>>>>;

175 // testing network type (replaced batch normalization with fixed affine transforms)
176 using anet_type = loss_metric<fc_no_bias<128,avg_pool_everything<
177     alevel0<
178     alevel1<
179     alevel2<
180     alevel3<
181     alevel4<
182     max_pool<3,3,2,2,relu<affine<con<32,7,7,2,2,
183     input_rgb_image
184     >>>>>>>>>>;

```

训练阶段创建，设置学习率和同步文件，并通过多个加载线程持续产生 mini-batch。每个 mini-batch 包含多个身份以及每个身份的多张图片，使模型可以同时学习同一身份和不同身份之间的距离关系。训练结束后，示例把网络清理并序列化保存为 `metric_network_renseset.dat`。

训练器初始化代码如下。这里的 设置了权重衰减和动量，设置初始学习率，则用于周期性同步训练状态。

```

218 net_type net;

219 dnn_trainer<net_type> trainer(net, sgd(0.0001, 0.9));
220 trainer.set_learning_rate(0.1);
221 trainer.be_verbose();
222 trainer.set_synchronization_file("face_metric_sync", std::chrono::minutes(5));
223 // Keep this small so the course experiment can finish in a reasonable time.
224 // For a real model, use much more data and a much larger threshold.
225 trainer.set_iteations_without_progress_threshold(80);
226 const unsigned long max_training_steps = 500;

```


数据加载线程通过 把图片和标签送入训练主线程。主线程只需要不断取出队列中的 batch 并调用，从而把耗时的图片读取、解码和增强工作与训练计算并行起来。

```

229 // If you have a lot of data then it might not be reasonable to load it all
230 // into RAM. So you will need to be sure you are decompressing your images
231 // and loading them fast enough to keep the GPU occupied. I like to do this
232 // using the following coding pattern: create a bunch of threads that dump
233 // mini-batches into dlib::pipes.
234 dlib::pipe<std::vector<matrix<rgb_pixel>>> qimages(4);
235 dlib::pipe<std::vector<unsigned long>> qlabels(4);
236 auto data_loader = [&qimages, &qlabels, &objs](time_t seed)
237 {
238     dlib::rand rnd(time(0)+seed);
239     std::vector<matrix<rgb_pixel>> images;
240     std::vector<unsigned long> labels;
241     while(qimages.is_enabled())
242     {
243         try
244         {
245             load_mini_batch(5, 5, rnd, objs, images, labels);
246             qimages.enqueue(images);
247             qlabels.enqueue(labels);
248         }
249         catch(std::exception& e)
250         {
251             cout << "EXCEPTION IN LOADING DATA" << endl;
252             cout << e.what() << endl;
253         }
254     }
255 };
256 // Run the data_loader from 5 threads. You should set the number of threads
257 // relative to the number of CPU cores you have.
258 std::thread data_loader1([data_loader]() { data_loader(1); });
259 std::thread data_loader2([data_loader]() { data_loader(2); });
260 std::thread data_loader3([data_loader]() { data_loader(3); });
261 std::thread data_loader4([data_loader]() { data_loader(4); });
262 std::thread data_loader5([data_loader]() { data_loader(5); });

263 // Here we do the training. The original example stops only after the learning
264 // rate becomes very small. For this course experiment, we also cap the number
265 // of training steps so the program reaches the save-and-verify stage quickly.
266 while(trainer.get_learning_rate() >= 1e-4 && trainer.get_train_one_step_calls() <
    ↪ max_training_steps)
267 {
268     qimages.dequeue(images);
269     qlabels.dequeue(labels);
270     trainer.train_one_step(images, labels);

```

学习率降到阈值以下后，示例保存网络并关闭加载线程。保存模型前调用 是为了去掉训练时的临时缓存，使序列化文件更适合后续推理使用。

```

274 cout << "training steps: " << trainer.get_train_one_step_calls() << endl;

275 // Wait for training threads to stop
276 trainer.get_net();
277 cout << "done training" << endl;

```

```
278 // Save the network to disk
279 net.clean();
280 serialize("metric_network_renset.dat") << net;

281 // stop all the data loading threads and wait for them to terminate.
282 qimages.disable();
283 qlabels.disable();
284 data_loader1.join();
285 data_loader2.join();
286 data_loader3.join();
```

训练完成后，示例把图片送入测试网络得到嵌入向量，并根据向量距离判断两张图片是否属于同一身份。这个过程与浏览器项目中的 思路一致：先得到人脸描述子，再依据距离阈值完成匹配。

```
295 // Now, just to show an example of how you would use the network, let's check how
296 ↪ well
297 // it performs on the training data.
298 dlib::rand rnd(time(0));
299 load_mini_batch(5, 5, rnd, objs, images, labels);

300 // Normally you would use the non-batch-normalized version of the network to do
301 // testing, which is what we do here.
302 anet_type testing_net = net;

303 // Run all the images through the network to get their vector embeddings.
304 std::vector<matrix<float,0,1>> embedded = testing_net(images);

305 // Now, check if the embedding puts images with the same labels near each other and
306 // images with different labels far apart.
307 int num_right = 0;
308 int num_wrong = 0;
309 for (size_t i = 0; i < embedded.size(); ++i)
310 {
311     for (size_t j = i+1; j < embedded.size(); ++j)
312     {
313         if (labels[i] == labels[j])
314         {
315             // The loss_metric layer will cause images with the same label to be
316             ↪ less
317             // than net.loss_details().get_distance_threshold() distance from each
318             // other. So we can use that distance value as our testing threshold.
319             if (length(embedded[i]-embedded[j]) <
320                 ↪ testing_net.loss_details().get_distance_threshold())
321                 ++num_right;
322             else
323                 ++num_wrong;
324         }
325     }
326     else
327     {
328         if (length(embedded[i]-embedded[j]) >=
329             ↪ testing_net.loss_details().get_distance_threshold())
330             ++num_right;
331         else
332             ++num_wrong;
333     }
334 }
```

```
329     }  
330 }  
  
331 cout << "num_right: "<< num_right << endl;  
332 cout << "num_wrong: "<< num_wrong << endl;
```

最后的两两比较代码尤其值得注意。它把同标签图片距离小于阈值视为正确,把不同标签图片距离大于等于阈值视为正确。这与浏览器端最近邻匹配的逻辑一致:模型负责输出描述子,识别逻辑负责比较描述子距离并决定是否接受某个身份标签。